Laravel

Rutas

Pepe Devesa

ies SERPIS | Departament Informàtica

Continguts

1 Pròleg **3**

2 Rutas simples **4**

3 Rutas con parámetros **4**

4 Validación de parámetros **5**

5 Rutas con nombre o named routes **5**

6 Verbos http: GET, POST, PUT,VIEW, PATCH y DELETE **6**

7 Combinación de elementos en rutas **6**

8 Ejercicios **7**

8.1 Definición de las rutas 7

8.2 Tablas de multiplicar 8

1 Pròleg

Las rutas `routes` son un mecanismo que permite a Laravel establecer qué respuesta enviar a una petición que intenta acceder a una determinada URL. Estas rutas se especifican en diferentes archivos dentro de la carpeta routes de nuestro proyecto Laravel.

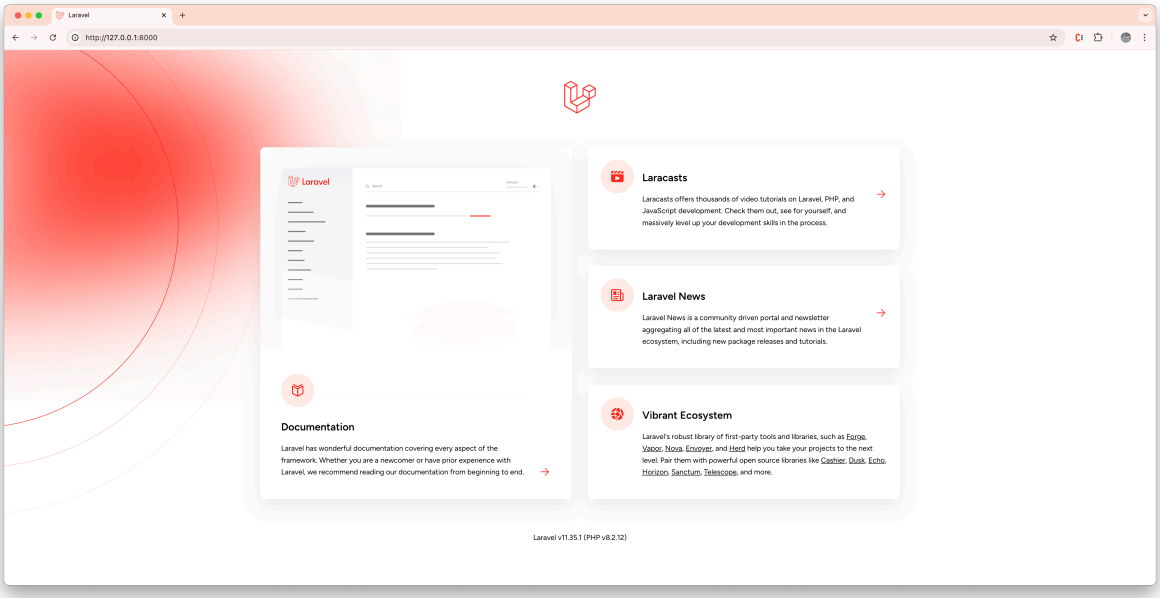
Podríamos decir que existen dos tipos principales de rutas:

- Las rutas `web` (almacenadas en el archivo `routes/web.php` de la aplicación), que nos permitirán cargar distintas vistas en función de la URL que indique el cliente.
- Las rutas `API` (almacenadas en el archivo `routes/api.php`), a través de las cuales definiremos distintos servicios `REST`, como veremos también más adelante.

Nos vamos a centrar durante esta sesión en el primer grupo, por lo que editaremos el contenido del archivo `routes/web.php`. Estas rutas son las más habituales, y se utilizan para recuperar contenidos típicamente en formato HTML.

Inicialmente ya existe una ruta predefinida hacia la raíz del proyecto, que carga la página de bienvenida al mismo:

```
Route::get('/', function () {
    return view('welcome');
});
```



(a) welcome

Para definir una ruta en Laravel, se hace una llamada a un método estático de la clase `Route` (en el ejemplo anterior, al método `Route::get`). Como primer parámetro, especificaremos la URL de la ruta (la ruta raíz, en el ejemplo anterior), y como segundo parámetro, la función que se va a ejecutar cuando algún cliente haga una petición a esa ruta.

2 Rutas simples

Las rutas simples tienen un nombre de ruta fijo, y una función que responde a dicho nombre emitiendo una respuesta. Un ejemplo es la ruta raíz que viene por defecto en nuestro proyecto. Podríamos definir otra ruta a continuación de esa, mediante la cual, si accedemos a la URL http://ip:8000/salut nos muestre un saludo.

```
    Route::get('salut', function()
    {
        return 'Hola món!';
    });
```

o nos muestre la fecha y hora actuales (http://ip:8000/fecha)

```
Route::get('fecha', function() {
   return date("d/m/y h:i:s");
    });
```

3 Rutas con parámetros

Es también posible pasar **parámetros en la URL** de la ruta. Para ello, incluimos el nombre del parámetro entre llaves, y lo pasamos también a la función del segundo parámetro. Por ejemplo, si definimos una ruta para saludar al nombre que nos llega como parámetro, el código quedaría así:

```
Route::get('salut/{nom}', function($nombre) {
   return "Hola, " . $nom;
   });
```

En este caso, si el parámetro es obligatorio y no lo indicamos en la URL, nos redirigirá a una página de error 404. Para indicar que un parámetro no es obligatorio, se termina su nombre con un interrogante, y también conviene darle un valor por defecto en la función PHP de respuesta. Así modificaríamos la ruta anterior para que el nombre del usuario sea opcional y, en caso de no ponerlo, se le asigne el nombre `Invitado`.

```
Route::get('saludo/{nombre?}', function($nombre = "Invitado") {
       return "Hola, " . $nombre;
```

```
    });
```

4 Validación de parámetros

Algunos parámetros será preciso que sigan un determinado patrón. Por ejemplo, un identificador numérico sólo contendrá dígitos. Para asegurarnos de eso, podemos emplear el método `where` al definir la ruta. A este método le pasamos dos parámetros: * el nombre del parámetro a validar, * y la expresión regular que tiene que cumplir.

En el caso del nombre anterior, si queremos que sólo contenga letras (mayúsculas o minúsculas), podemos hacer algo así:

```
Route::get('saludo/{nombre?}', function($nombre = "Invitado") {
   return "Hola, " . $nombre;
})->where('nombre', "[A-Za-z]+");
```

5 Rutas con nombre o named routes

En ocasiones puede ser conveniente asociar un nombre a una ruta. Especialmente, **cuando esa ruta va a formar parte de un enlace en alguna página de nuestro sitio**, ya que en un futuro la ruta podría cambiar, y de este modo evitamos tener que actualizar los enlaces con el nuevo nombre. Para ello, al definir la ruta, le asociamos con la función `name` el nombre que queramos. Por ejemplo:

```
Route::get('contacto', function() {
   return "Página de contacto";
})->name('ruta_contacto');
```

Ahora, si queremos definir un enlace a esta ruta en cualquier parte, basta con emplear la función route de Laravel, indicando el nombre que le hemos asignado a esta ruta. Por lo tanto, en lugar de poner esto:

```
echo '<a href="/contacto">Contacto</a>';
```

Podemos hacer algo como esto otro, tal y como veremos más adelante cuando definamos nuestras vistas:

```
<a href="{{ route('ruta_contacto') }}">Contacto</a>
```

De este modo, ante futuros cambios en las rutas, sólo deberemos cambiar la URL en el correspondiente `Route::get` de `routes/web.php`. Por Ejemplo:

En el futuro se puede cambiar la forma en la que lo ve el usuario. Por ejemplo, puedes cambiar `Route::get('/contactame'...)` por `Route::get('/acerca-de'...)`, pero no tendrás que modificar ni una línea de código más en tu aplicación, porque internamente esa ruta seguirá llamándose route('ruta_contacto').

6 Verbos http: GET, POST, PUT,VIEW, PATCH y DELETE

Además de GET, en el enrutador se pueden enrutar otras acciones:

```
Route::get();    // Solicitudes habituales
Route::post();   // Recepción de datos de formulario (para insert)
Route::put();    // Recepción de datos para update (también puede
   escribirse Route::patch(), que no es lo mismo, pero casi)
Route::view();   //Este método es importante por simplicidad. Evita tener
   que pasar por un controlador si solo queremos devolver una vista simple
    sin lógica adicional.
Route::delete(); // Recepción de datos para delete
Route::match(array('GET','POST'), 'ruta', acción)
                 // Responderá tanto a GET como a POST
```



Los verbos PUT, PATCH y DELETE no están soportados aún por HTML. Las razones por las que esto ocurre son tan confusas que, sinceramente, creo que ni siquiera en el W3C lo entienden. La cuestión es que aún no existen, y punto.

7 Combinación de elementos en rutas

Podemos combinar varias cláusulas `where` en una ruta para validar distintos parámetros que pueda tener, y también enlazar estas llamadas con una a la función `name` para nombrar la ruta. Por ejemplo, la siguiente ruta **espera recibir un nombre con caracteres, y un id numérico**, ambos con valores por defecto:

```
Route::get('saludo/{nombre?}/{id?}', function($nombre="Invitado", $id=0) {
   return "Hola $nombre, tu código es el $id";
   })->where('nombre', "[A-Za-z]+")
   ->where('id', "[0-9]+")
   ->name('saludo');
```

Si accedemos a cada una de las siguientes URLs, obtendremos cada una de las respuestas indicadas:

Ruta	Respuesta
/saludo	Hola Invitado, tu código es el 0
/saludo/Pepe	Hola Pepe, tu código es el 0
/saludo/Pepe/3	Hola Pepe, tu código es el 3
/saludo/3	ERROR 404 (URL incorrecto)

Notar que el último caso es incorrecto. No podemos especificar un id sin haber especificado un nombre delante, porque incumple el patrón de la URL. Se puede dejar un parámetro omitido, siempre y cuando los posteriores también lo estén.

8 Ejercicios

8.1 Definición de las rutas

En este ejercicio vamos a definir las rutas principales que va a tener nuestro sitio web.

Para empezar simplemente indicaremos que las rutas devuelvan una cadena (así podremos comprobar que se han creado correctamente). A continuación se incluye una tabla con las rutas a definir (todas de tipo GET) y el texto que tienen que mostrar:

Ruta	Texto a mostrar
/	Pantalla Principal
login	login usuario
logout	logout usuario
catalog	Listado de peliculas
catalog/show/{id}	Vista detalle película {id}
catalog/create	Añadir Película
catalog/edit/{id}	Modificar película {id}

Para comprobar que las rutas se hayan creado correctamente utiliza el comando de artisan que devuelve un listado de rutas y además prueba también las rutas en el navegador.

```
php artisan route:list
```

8.2 Tablas de multiplicar

Hacer una ruta **/tabla/{numero}** que me devuelva la tabla de multipplicar del número que se le pasa, tiene que ser un número, en caso de estar vacio el número sacará la tabla del 3.